03 SERVICES (PROVIDERS) IN NESTJS — THEORY

A Service is a class that holds the actual business logic of your application — database calls, calculations, external API calls, rules — everything a Controller should NOT contain. In NestJS, a service is one kind of "Provider" (the general term for anything Nest can create and inject via Dependency Injection).

```
Controller -> (calls) -> Service -> (talks to) -> Database / APIs / etc.
```

1. DECLARING A SERVICE

```ts
import { Injectable } from '@nestjs/common';

@Injectable()
export class CatsService {
  private cats: Cat[] = [];

  findAll(): Cat[] {
    return this.cats;
  }
}
```

- @Injectable() marks this class as something Nest's DI container is allowed to create and hand out to whoever asks for it.
- Without @Injectable(), Nest won't manage this class — you'd have to instantiate it manually with "new", losing all the benefits below.

2. REGISTERING IT

A service must be listed in a module's "providers" array so Nest knows it exists and can inject it:

```ts
import { Module } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class CatsModule {}
```

Nest creates exactly ONE instance of CatsService (by default — this is called "singleton scope") and reuses it everywhere it's injected.

3. INJECTING IT INTO A CONTROLLER (DEPENDENCY INJECTION)

```ts
@Controller('cats')
export class CatsController {
  constructor(private readonly catsService: CatsService) {}
```

```
  @Get()
  findAll() {
    return this.catsService.findAll();
  }
}
```

- Nest sees CatsService as a constructor parameter and automatically creates/passes in an instance — you never write "new CatsService()" yourself.
- "private readonly catsService" is TypeScript shorthand that both declares a class property AND assigns the constructor argument to it in one line.
- This is Dependency Injection (DI): classes declare WHAT they need, and Nest figures out HOW to provide it.

4. A MORE REALISTIC SERVICE (WITH LOGIC)

```
import { Injectable, NotFoundException } from '@nestjs/common';

@Injectable()
export class CatsService {
  private cats: Cat[] = [];

  findAll(breed?: string): Cat[] {
    return breed
      ? this.cats.filter((c) => c.breed === breed)
      : this.cats;
  }

  findOne(id: string): Cat {
    const cat = this.cats.find((c) => c.id === id);
    if (!cat) {
      throw new NotFoundException(`Cat ${id} not found`);
    }
    return cat;
  }

  create(dto: CreateCatDto): Cat {
    const cat: Cat = { id: Date.now().toString(), ...dto };
    this.cats.push(cat);
    return cat;
  }

  update(id: string, dto: UpdateCatDto): Cat {
    const cat = this.findOne(id);
    Object.assign(cat, dto);
    return cat;
  }

  remove(id: string): void {
    const index = this.cats.findIndex((c) => c.id === id);
    if (index === -1) {
      throw new NotFoundException(`Cat ${id} not found`);
    }
    this.cats.splice(index, 1);
  }
}
```

Notice: filtering, validation, "not found" rules, and ID generation all live HERE — not in the controller. Thrown exceptions like NotFoundException are automatically caught by Nest and turned into the correct HTTP status response (404, in this case).

5. SERVICES CAN INJECT OTHER SERVICES/PROVIDERS

Services aren't just called by controllers — they can depend on other providers too (a database repository, a config service, another domain's service, a logger):

```
@Injectable()
export class CatsService {
  constructor(
    @InjectRepository(Cat) private catsRepo: Repository<Cat>,
    private readonly configService: ConfigService,
  ) {}

  findAll() {
    return this.catsRepo.find();
  }
}
```

This is how real apps connect to an actual database (e.g. via TypeORM's Repository) instead of an in-memory array.

6. WHY LOGIC LIVES HERE, NOT IN THE CONTROLLER

- Testability — you can unit test CatsService directly (no HTTP, no Express) by just calling its methods and checking return values.
- Reuse — the same CatsService can be called from a different controller, a CLI script, a cron job, or a GraphQL resolver, without duplicating the logic.
- Separation of concerns — the controller only knows about HTTP (routes, status codes); the service only knows about the domain (cats, rules about cats). Neither needs to know about the other's details.

7. PROVIDER SCOPES (ADVANCED, GOOD TO KNOW)

By default every provider is a singleton — one shared instance for the whole app's lifetime. You can change this with @Injectable({ scope }):

Scope.DEFAULT	One instance, shared everywhere (default, fastest).
Scope.REQUEST	A new instance created per incoming request (useful if you need per-request state, e.g. the current user) — has a small performance cost.
Scope.TRANSIENT	A new instance every time it's injected anywhere.

Stick with the default unless you have a specific reason not to — it's simpler and faster.

★ RULE OF THUMB

If it's a rule about the data (validation beyond simple shape checks, calculations, what counts as "not found", how two records relate) — it belongs in a service. If it's about HTTP (what URL, what status code, what headers) — it belongs in the controller.

NEST CLI — GENERATING A SERVICE

Instead of writing the file by hand, the Nest CLI can scaffold it:

```
nest generate service <name>
nest g s <name>              (short form)
```

Example:

```
nest g s user
```

This creates:

src/user/user.service.ts	— the @Injectable() class itself
src/user/user.service.spec.ts	— a starter unit test file

It ALSO automatically registers the new service in the nearest module's "providers" array (e.g. adds it to user.module.ts if one exists in that folder) — saving you the manual wiring step shown earlier in this file.

Useful variations:

nest g s user --no-spec	Skip generating the .spec.ts test file
nest g s user/admin	Nest the service under a subfolder (src/user/admin/admin.service.ts)
nest g resource user	Generates an ENTIRE CRUD resource at once: module + controller + service + DTOs + entity — useful when starting a brand-new feature from scratch.

Note: if you're adding a service into an existing folder that has NO user.module.ts (like this project's src/user/ folder, where UserController/UserService are registered directly in the root app.module.ts instead), the CLI will still generate the file correctly, but you may need to manually add it to the correct module's "providers" array yourself — check app.module.ts (or the relevant module) after running the command.